Step 3: create constraints (choose your path)

iCE40 path (PCF)

Create `constraints/board.pcf`:

```
# Replace <LED_PIN> and <CLK_PIN> with your board's values
set_io led <LED_PIN>
set_io clk <CLK_PIN>

# If you have a reset button:
# set_io rst_n <RST_PIN>
```

PCF usage with `--pcf` is explicitly documented for iCE40 in nextpnr docs.

ECP5 path (LPF)

Create `constraints/board.lpf`:

```
# Replace <SITE_NAME> with your board's LED site
LOCATE COMP "led" SITE "<LED_SITE>";
IOBUF PORT "led" IO_TYPE=LVCMOS33;

# Replace <CLK_SITE> with your clock site
LOCATE COMP "clk" SITE "<CLK_SITE>";
IOBUF PORT "clk" IO_TYPE=LVCMOS33;

# Optional reset mapping similarly
```

LPF support via `--lpf` is a documented nextpnr-ecp5 option.

Important: clock pins may need special treatment on some boards (dedicated clock pins). Your board docs will tell you.

Build commands: iCE40 (Yosys → nextpnr-ice40 → icepack)

This section assumes your iCE40 device is something like HX1K/HX8K/UP5K etc. (Board docs will specify part/package.)

Step-by-step: build bitstream for iCE40

1. Synthesize with Yosys using `synth_ice40`:
   - Yosys provides `synth_ice40` as a target-specific synthesis command.

   Example:

   ```
   yosys -p "read_verilog src/blinky.v; synth_ice40 -top blinky -json build/blinky.json"
   ```

2. Place & route with nextpnr-ice40:
   - nextpnr's iCE40 docs describe PCF constraints (`--pcf`).

   Example:

   ```
   nextpnr-ice40 --hx8k --package ct256 \
     --json build/blinky.json \
     --pcf constraints/board.pcf \
     --asc build/blinky.asc
   ```

   (Replace `--hx8k --package ct256` with your device/package.)

12 COMMON REASONS FPGA BUILDS FAIL

1. **Wrong device/package flag** in nextpnr
2. **Wrong constraints file** or wrong pin names
3. **Top module mismatch** (your build targets the wrong module)
4. **Clock pin not constrained** properly
5. **Reset polarity confusion** (`rst_n` active low but wired opposite)
6. **LED is active-low** on some boards (so your logic appears inverted)
7. **No actual clock** (board oscillator not mapped / wrong pin)
8. **USB permissions / drivers** for programming cable
9. **Programming tool expects different file format**
10. **Multiple drivers on a net** (bad HDL wiring)
11. **Floating inputs** (buttons need pull-ups/pull-downs)
12. **You changed constraints but didn't rebuild cleanly**

When in doubt: simplify.

- one LED
- one clock
- no reset
- minimal module

Then add complexity.

3. Pack bitstream:

Project IceStorm tooling provides the fully open-source iCE40 flow, including converting the ASCII config to a bitstream format used for programming.

Example:

```
icepack build/blinky.asc build/blinky.bin
```

At this point you have a `blinky.bin` ready to upload.

Build commands: ECP5 (Yosys → nextpnr-ecp5 → bitstream)

ECP5 flow uses Project Trellis for the architecture/bitstream documentation and nextpnr-ecp5 for P&R.

Step-by-step: build bitstream for ECP5

1. Synthesize with Yosys:

   Yosys provides `synth_ecp5` for ECP5 targets.

   Example:

   ```
   yosys -p "read_verilog src/blinky.v; synth_ecp5 -top blinky -json build/blinky.json"
   ```